

Unveiling malicious DNS behavior profiling and generating benchmark dataset through application layer traffic analysis

MohammadMoein Shafi ^{a,*}, Arash Habibi Lashkari ^{a,b}, Hardhik Mohanty ^b

^a Department of Electrical Engineering and Computer Science, York University, Toronto, Ontario, Canada

^b Behaviour-Centric Cybersecurity Center (BCCC), School of Information Technology, York University, Toronto, Ontario, Canada

ARTICLE INFO

Keywords:

DNS analysis
Behavioral profiling
Application layer security
Anomaly detection
Malicious behavior identification
Pattern extraction
Behavior similarity calculation
ALFlowLyzer
BCCC-CIC-bell-DNS-2024

ABSTRACT

The Domain Name System (DNS) is a prime target for cyber attacks, necessitating the monitoring and analysis of DNS activities to detect malicious behaviors. This paper presents an innovative DNS behavioral profiling approach that addresses challenges posed by the dynamic landscape of cyber threats, encompassing issues like evasion tactics, content variability, discerning malicious intent, navigating URL obfuscation, low and slow tactics, and maintaining accuracy in the face of diverse normal behaviors, contributing to the advancement of robust threat detection. The framework leverages unique feature behaviors and correlations, incorporating a novel feature selection algorithm, pattern extraction methodology, and a robust neural network architecture for accurate profile construction. The research also includes the development of ALFlowLyzer, a custom application layer network flow analyzer, and introduces the BCCC-CIC-Bell-DNS-2024 dataset, addressing limitations in widely used public DNS datasets. Experimental results demonstrate the effectiveness of the proposed model in profiling various DNS activities.

1. Introduction

DNS serves as the internet's equivalent of an address book, converting human-readable domain names into machine-readable IP addresses. Because DNS plays a pivotal role in numerous applications, organizations commonly allow DNS traffic through UDP port 53, which unfortunately makes it a prime target for cyber attacks. These attacks encompass a range of malicious activities such as Distributed Denial of Service (DDoS), DNS Amplification, Exfiltration, Hijacking, Tunneling, Poisoning, etc. Consequently, it becomes imperative to monitor DNS activities rigorously to detect any anomalies or potential threats [1]. However, detecting and mitigating such attacks present significant challenges due to their intricate and ever-evolving nature. Traditional detection methods often struggle to promptly and accurately identify and halt these attacks in real-time. Moreover, with cyber attackers continuously refining their tactics, conventional detection mechanisms increasingly fail to recognize newly emerging variants of DNS attacks. Hence, there is a growing acknowledgment of the necessity to harness Artificial Intelligence (AI) techniques to bolster DDoS detection capabilities [2].

AI-driven techniques such as Machine Learning (ML) and Deep Learning (DL) algorithms heavily depend on data to understand and differentiate different DNS behaviors linked with diverse activities. Nonetheless, the efficacy of these detection approaches greatly depends on the presence of extensive and high-quality datasets for training and assessing purposes. Hence, the key to enhancing the capabilities of any detection methods primarily lies in having dependable and thorough evaluation datasets [2].

Developing datasets for DNS detection involves a complex procedure that necessitates careful planning, precise execution, and specialized knowledge. Building the framework for dataset creation involves establishing a controlled environment that

* Corresponding author.

E-mail address: moeinsh@yorku.ca (M. Shafi).

<https://doi.org/10.1016/j.compeleceng.2024.109436>

Received 12 January 2024; Received in revised form 23 May 2024; Accepted 22 June 2024

Available online 5 July 2024

0045-7906/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

faithfully replicates real-world network scenarios, focusing on scalability, reliability, and security. This undertaking demands in-depth expertise in network architecture and system administration. Thorough analysis is essential to derive valuable insights and verify the dataset's quality, utilizing sophisticated anomaly detection and classification algorithms. Thus, assuring the integrity and thoroughness of the dataset creation process is vital for progressing DNS detection capabilities and improving DNS security [1].

This study delves into the essential realm of establishing a thorough DNS dataset, highlighting the crucial significance of high-quality datasets in bolstering the efficiency of DNS detection and characterization algorithms and systems.

By thoroughly examining the most recent publicly accessible DNS datasets, we pinpointed their limitations across multiple aspects and discerned a prospect for dataset integration and improvement. By elucidating the obstacles and challenges present in current datasets, this study aims to lay the groundwork for generating more dependable and inclusive DNS datasets by integrating, cleaning, and improving preceding datasets. This endeavor ultimately propels the advancement of cutting-edge DNS detection and mitigation methods.

Building upon this foundational understanding, the first contribution in this work is the introduction of a new DNS dataset named *BCCC-CIC-Bell-DNS-2024*. This dataset is a result of the integration, cleaning, and improvement of two previous datasets named *CIC-Bell-DNS-2021* [3] and *CIC-Bell-DNS-EXF-2021* [4]. The development of this dataset involves a comprehensive examination of two datasets' raw data for integration, cleaning, and precise labeling. Additionally, extensive research was conducted on DNS analysis, resulting in the introduction of a new concept termed "DNS Flow" alongside an extensive feature set comprising over 120 features. Furthermore, a specialized Application Layer Flow Analyzer software (ALFlowLyzer) was designed and developed, aligning with the DNS Flow concept and the corresponding feature set, to generate CSV files from the datasets' raw data.

Next to dataset creation, this research proposes a DNS behavior profiling model to detect different DNS activities. Behavioral profiling proves effective in constructing standard models for network entities, facilitating anomaly detection through deviations from expected behavior [2]. The proposed model is rooted in two principles: (1) the distinctive nature of feature behavior across diverse activities and (2) the dynamic correlations among features within these activities. By harnessing these principles, precise profiles are systematically formulated and validated through empirical experimentation. A novel feature selection algorithm is employed to discern optimal features based on behavioral analysis, while the pattern extraction phase captures intricate interdependencies among these features values. In the final phase, a neural network architecture is deployed to ascertain the weights associated with each profile derived from the extracted patterns of individual DNS activities. Additionally, a metric called "behavior similarity" quantifies the similarity between activities, facilitating comparison and clustering.

Finally, the experimental results conducted with the newly created dataset confirm the effectiveness of the proposed detection model in precisely recognizing diverse DNS actions. The comparison with previous works demonstrates the superior performance of the proposed model in detecting malicious DNS activities.

This paper contributes to DNS behavior profiling through the following key innovations:

- Introduction of a new graph-based feature selection algorithm.
- Introduction of a behavior similarity calculation algorithm.
- Introduction of DNS Flow as a vital component in DNS analysis.
- Creation of a new and sophisticated behavioral profiling system through pattern extraction and neural network architecture.
- Implementation of ALFlowLyzer, an application layer network flow analyzer with more than 120 DNS features.
- Introduction of a new malicious DNS dataset, named *BCCC-CIC-Bell-DNS-2024*, with a DNS flow-based approach.

The remaining part of this paper is structured as follows: Section 2 provides an overview of previous research on analyzing malicious DNS traffic. Section 3 presents a detailed explanation of the proposed profiling model. Section 4 introduces ALFlowLyzer. Section 5 explores the available datasets and discusses the decision to integrate two datasets, along with an explanation of the characteristics of the new dataset. Section 6 presents the experimental setup and the results of applying the proposed model to the newly introduced dataset. Section 7 analyzes and discusses these results, emphasizing the key findings. Lastly, Section 8, concludes the paper and suggests potential directions for future research.

2. Related works

This section provides an overview of prior research on analyzing malicious DNS traffic. The previous works are categorized based on their primary methodologies, namely rule-based, learning-based, and profiling-based methods. It concludes by highlighting the limitations of previous research studies and identifying the challenges this work aims to address.

2.1. Rule-based approaches

Rule-based methods analyze DNS traffic features through rule formulation and comparing features with predefined signatures. In the work by [5], they proposed a collaborative framework to find event correlation among standard malicious URLs in Android devices. This method is used to pinpoint the malware attacks in several unmonitored smartphones in the wireless cellular system.

Similarly [6] analyzed the use of the DNS in detecting domains and channels that are used for distributing malicious payloads. The research characterized the malicious payload distribution channels by analyzing passive DNS traffic and modeled the DNS query and response patterns used during malicious payload distribution.

In addition to these works [7], introduced a rule-based approach known as DNS-BD (DNS Botnet Detection) aimed at enhancing the accuracy of botnet detection through DNS traffic analysis. This method relies on identifying anomalous DNS query and response behaviors using predefined rules designed for DNS queries and responses.

2.2. Learning-based approaches

Learning-based approaches utilize ML and DL techniques to analyze DNS traffic patterns and identify potential threats. In the work by Nguyen [8], they proposed a behavioral analysis approach based on unsupervised machine learning. Furthermore, this work compared the performance of four algorithms, K-means, GMM, LOF, and DBSCAN, on the dataset records with different attack types. Finally, the authors combined DBSCAN and KNN to achieve better performance.

Likewise [9] proposed an improved version of EasyEnsemble, named HAC-EasyEnsemble for detecting malicious domains. The proposed framework effectively dealt with the within-class imbalance problem in tandem with the between-class imbalance problem.

Following a similar approach [10] introduced a method for detecting DNS tunnels based on multidimensional analysis. The random forest algorithm served as the classification model, screens out the suspicious packets that may belong to the DNS tunnel from the DNS query packets. This packet detection technique, combined with an anomaly detection model based on a one-class support vector machine, successfully identified abnormal DNS session traffic.

There are also a few works that apply DL algorithms for malicious DNS traffic detection. As an instance [11] proposed a detection model named LSTM-AE based on domain query and time-series features. It integrated LSTM-based time-series characterization and unsupervised autoencoder to detect data leakage malware.

Likewise [12] proposed a fusion of two classifiers to reduce the false positive rate in DNS tunneling (DNST) detection. DNST is utilized for both data Exfiltration and Benign purposes. The dual property of DNST makes it challenging to segregate malicious DNST traffic from normal traffic. The classifiers used in this paper are Convolution Neural Networks (CNN) and random forest (RF). The DNS query names are fed as input to the CNN classifier, which produces the class probabilities. Next, the average class probability and 11 other features are input to the RF classifier.

2.3. Behavioral profiling-based approaches

Profiling in DNS traffic analysis involves systematically collecting and analyzing distinctive features and behavioral patterns. It includes creating models to characterize normal DNS traffic, enabling the identification of anomalies or malicious activities based on deviations from established norms.

In the work by [13], authors present a three-phase model for DNS service profiling, using optimal entropy-based feature selection, Holt Winter's method for normal behavior prediction, and upper and lower thresholds for anomaly detection. The normal profile is regularly updated based on predicted and actual behaviors.

In a separate study conducted by [14], a pioneering DNS-based profiling scheme was introduced using real datasets of Mirai-like botnet activity observed across globally distributed honeypots. The authors initially examine the features traditionally employed for botnet profiling and illustrate the potential enhancements in profiling IoT-based botnets by harnessing DNS data from a single DNS record. Additionally, they evaluate the efficacy of the developed feature set across different ML classifiers, showcasing the practical utility of their approach.

In a similar work, [15] introduced IoTFinder, a system tailored for the efficient, large-scale passive detection of IoT devices. The approach capitalizes on distributed passive DNS data collection and devises a machine learning-driven framework to accurately identify a diverse array of IoT devices solely based on their DNS fingerprints. IoTFinder is designed as a multi-label classifier, and its accuracy is evaluated across various scenarios.

2.4. Synthesis

The previous works have exhibited several limitations, which can be summarized as follows:

1. Necessity of huge computational resources.
2. Low accuracy and high false positive rate.
3. Lack of a comprehensive dataset.
4. Lack of comprehensive feature set.
5. Restrictiveness to certain malicious DNS activities.
6. Limited ability to detect zero-day attacks.
7. Necessity for regularly updating the blacklist.
8. Necessity of prior knowledge to update activities.
9. Lack of real-time intrusion detection.
10. Susceptible to evasion tactics by attackers with knowledge of the system.

The proposed technique employs behavioral profiling to model malicious and benign DNS traffic based on the features extracted from the application layer meta-data and network traffic flow. In this study, we focus on addressing the first seven issues outlined above.

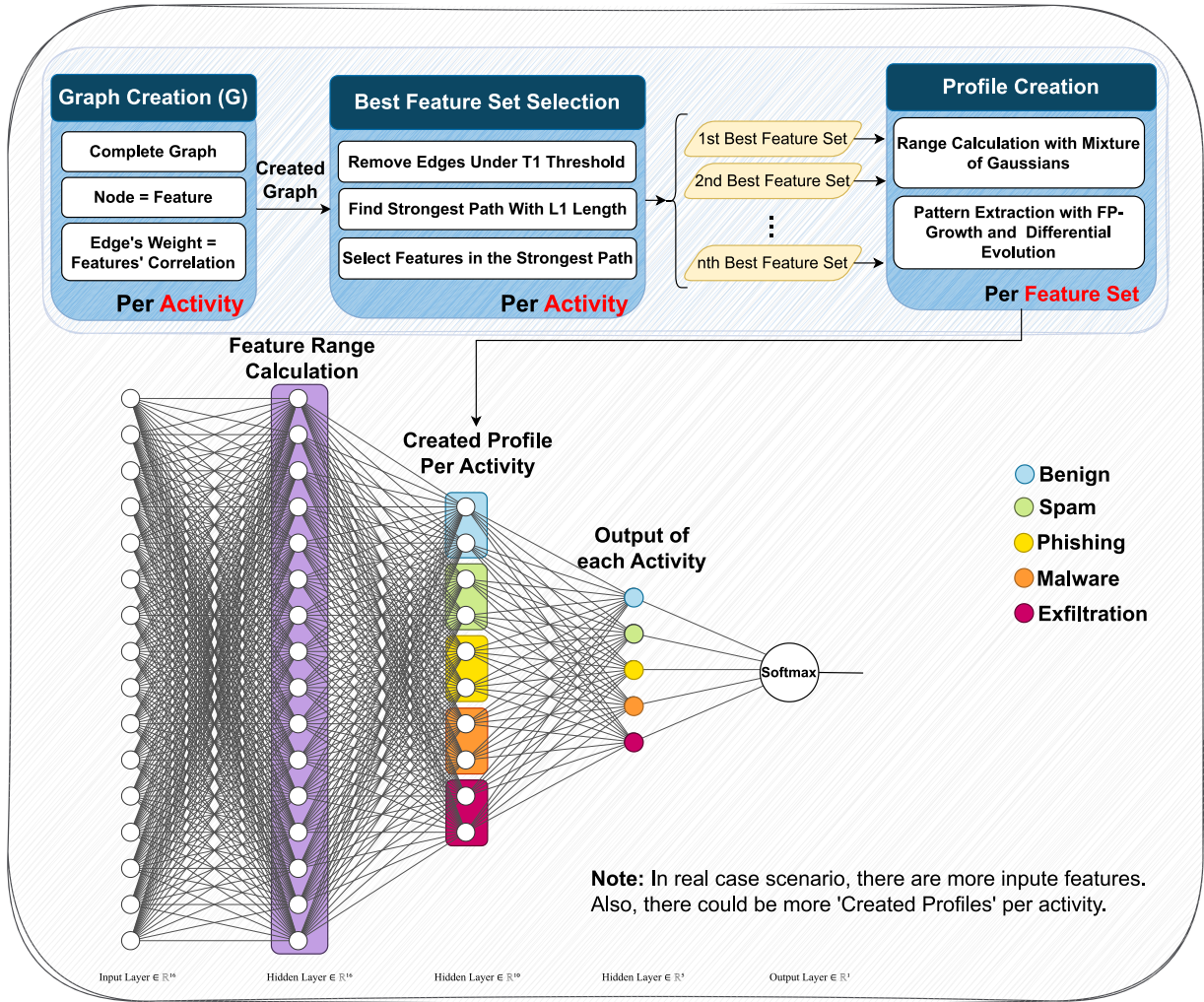


Fig. 1. General profiling module.

3. Proposed model

This section introduces an innovative behavioral profiling model for network traffic analysis focusing on DNS-based activities. The model enhances accuracy, interpretability, and efficiency by addressing identified limitations in current profiling methodologies. Fig. 1 outlines the general procedure, starting with feature selection for profile representation. A pattern extraction system then captures behavior patterns, forming the model's core. The final step involves a Neural Network structure, aggregating profiles, and assigning weights to each created profile to label new instances accurately. A similarity calculation algorithm derived from the feature selection process was also introduced to enhance data analysis. In the following subsections, a thorough examination of each step is provided.

3.1. Feature selection

Effective feature selection is crucial for constructing accurate profiles, forming the foundation for subsequent modeling stages. This section introduces a novel approach for optimizing features in DNS protocol analysis, utilizing a correlation graph to map and evaluate feature interrelationships.

This method entails creating a fully connected graph with initial edge weights of zero, which are updated based on feature correlations using Pearson's Correlation Coefficient [16]. The resulting weighted graph ($G = (V, E)$) is pruned by removing edges below a specified threshold, producing distinct correlation graphs for each activity label in the dataset (Fig. 5). To identify optimal feature sets, we traverse the correlation graph to find the most robust path of predefined length $(n + 1)$ between any two nodes (the path contains n different nodes). This departure from generic feature sets recognizes the activity-specific nature of profiling.

Finally, it is worth noting that we have rigorously evaluated different correlation algorithms, determining Pearson's Correlation Coefficient as the most suitable for this dataset and objectives considering the accuracy and efficiency. Experimental outcomes, detailed in Section 6 and illustrated in Fig. 6, validate this choice.

3.2. Profile creation

In this subsection, we explain the process involved in creating a profile based on the selected feature set for a given activity. The procedure initiates with the calculation of ranges for each selected feature, followed by utilizing the mapped values of these features during the pattern extraction phase to construct the profile. Subsequently, all the profiles generated for various activities are consolidated within a neural network structure, forming the foundation of the profiling system.

3.2.1. Range calculation

This subsection centers on the first concept, highlighting the distinct behavior displayed by each feature in various activities. For instance, in the *Benign* activity, the domain length must not surpass 10, whereas in the *Spam* activity, it should not exceed 20 (values provided are for illustration purposes). To put this into practice, the objective is to assess the potential intervals within which each attribute can operate in a specific activity. In the profiling context, the approach entails thoroughly examining these ranges for each feature, indicating the valid ranges of an individual feature within the designated activity. To identify these ranges, we employ the Mixture of Gaussians (MoG) function. MoG is a probabilistic model that represents the data distribution as a weighted sum of multiple Gaussian distributions. For one-dimensional data, the core equation encapsulating the MoG algorithm is formulated as follows [17]:

$$\hat{f}(x) = \sum_{k=1}^K w_k \mathcal{N}(x; \mu_k, \sigma_k^2) \quad (1)$$

where $\hat{f}(x)$ denotes the estimated density at point x , and K is the number of Gaussian components. Each component is characterized by its weight w_k , mean μ_k , and variance σ_k^2 . The Gaussian distribution $\mathcal{N}(x; \mu_k, \sigma_k^2)$ is given by:

$$\mathcal{N}(x; \mu_k, \sigma_k^2) = \frac{1}{\sqrt{2\pi}\sigma_k} e^{-\frac{1}{2}\left(\frac{x-\mu_k}{\sigma_k}\right)^2} \quad (2)$$

It is a probability density function describing the likelihood of x in the context of the k th Gaussian, with parameters μ_k and σ_k^2 representing mean and variance, respectively.

The likelihood function is maximized to optimize the process for determining the best set of parameters Θ^* . The likelihood function for MoG is given by [18]:

$$L(\Theta) = \prod_{i=1}^n \sum_{k=1}^K w_k \mathcal{N}(x_i; \mu_k, \sigma_k^2) \quad (3)$$

Here, Θ represents the set of parameters, including weights, means, and variances, and n is the total number of data points.

The optimization problem is typically formulated as maximizing the log-likelihood, equivalent to minimizing the negative log-likelihood. Thus, the optimization objective is:

$$\Theta^* = \arg \max_{\Theta} \sum_{i=1}^n \log \left(\sum_{k=1}^K w_k \mathcal{N}(x_i; \mu_k, \sigma_k^2) \right) \quad (4)$$

By iteratively updating the parameters through the Expectation-Maximization (EM) algorithm and maximizing the log-likelihood, the resulting Θ^* represents the parameters that best capture the underlying distribution of the one-dimensional data [19].

The optimization (EM algorithm) involves an initialization step, where initial guesses for the parameters are provided. The Expectation (E-step) and Maximization (M-step) constitute the iterative cycle. In the E-step, the responsibility r_{ik} of the i th data point for the k th Gaussian component is calculated:

$$r_{ik} = \frac{w_k \mathcal{N}(x_i; \mu_k, \sigma_k^2)}{\sum_{j=1}^K w_j \mathcal{N}(x_i; \mu_j, \sigma_j^2)} \quad (5)$$

This reflects the likelihood that data point x_i belongs to the k th Gaussian, considering the current parameter estimates. In the M-step, the parameters are updated to maximize the likelihood of the observed data. The updates are as follows:

- Update weights w_k :

$$w_k = \frac{\sum_{i=1}^n r_{ik}}{n} \quad (6)$$

- Update means μ_k :

$$\mu_k = \frac{\sum_{i=1}^n r_{ik} x_i}{\sum_{i=1}^n r_{ik}} \quad (7)$$

- Update variances σ_k^2 :

$$\sigma_k^2 = \frac{\sum_{i=1}^n r_{ik}(x_i - \mu_k)^2}{\sum_{i=1}^n r_{ik}} \quad (8)$$

The EM algorithm iterates between the E-step and M-step until convergence, where the likelihood of the data no longer increases significantly.

This comprehensive framework ensures the iterative optimization of the MoG model's parameters, providing an accurate representation of the underlying distribution of the one-dimensional data. The introduced formulas capture essential steps of the EM algorithm, and the associated parameters play crucial roles in determining the weights, means, and variances for each Gaussian component.

3.2.2. Pattern extraction

In this subsection, we conclude the creation of behavior profiles by incorporating the second concept elucidated earlier, which pertains to the diverse correlations observed among different features across various activities. As an illustrative example, considering features such as “domain length” and “numerical percentage”, in benign activities, these two features exhibit no meaningful correlation. However, in a Spam activity, they demonstrate a high positive correlation. This subsection delves into the intricacies of applying these feature correlations to the previously explained ranges. In essence, we seek to determine the associations between different ranges within the selected features. To elaborate, when the value of “domain length” falls within “range1”, what are the plausible values for “numerical percentage” in any activity? Notably, these new correlations, or rather, possibilities, may differ for each activity compared to another. The pattern extraction process herein unveils these latent behavioral nuances unique to each activity.

A crucial consideration is that the novel feature selection algorithm thoughtfully identifies the most correlated features. Consequently, this stage of profile creation invariably yields meaningful and rational patterns. This underscores the criticality of the feature selection algorithm, for it ensures that the pattern extraction process is connected to the underlying correlations between the features' values. Indeed, the successful identification of patterns hinges on the existence of substantial correlations between the features. As such, if the selected features lack meaningful correlations due to common algorithms like information gain, the ensuing pattern extraction process likely yields no meaningful patterns based on the features' value variations.

We employ Association Rule Mining algorithms to extract the diverse possibilities between different feature values within an activity. This family of algorithms aligns seamlessly with the second concept, as previously discussed. After a meticulous evaluation of various algorithms within this family, we have chosen to utilize the FP-Growth algorithm due to its proven efficacy in extracting association rules. However, this algorithm requires the specification of a minimum support threshold [20].

The minimum support threshold represents the minimum proportion of transactions required for a pattern to be considered. In the pursuit of identifying the optimal value for this parameter, we employed a rigorous approach involving the utilization of the Differential Evolution (DE) algorithm [21].

In the context of parameter tuning for the FP-Growth algorithm, DE emerges as a robust choice due to its adaptability to continuous parameter spaces and its effectiveness in navigating complex, nonlinear landscapes. DE operates on a population of candidate solutions, utilizing mutation, crossover, and selection operations to refine parameter sets iteratively. Given that the FP-Growth algorithm often involves a continuous parameter space, DE's capacity to efficiently explore such spaces while handling constraints aligns well with the requirements of the optimization task. Furthermore, its evolutionary nature allows it to discover diverse and potentially optimal configurations, making DE a suitable candidate for fine-tuning the parameters of FP-Growth and enhancing its overall performance.

When comparing DE with alternative optimization algorithms for parameter tuning in conjunction with FP-Growth, DE stands out for its versatility in handling continuous parameter spaces and its capability to explore complex search landscapes efficiently. In contrast to methods like grid search, which can be computationally expensive and impractical for continuous spaces, DE strikes a balance between exploration and exploitation. Its adaptability to diverse optimization scenarios, including those involving constraints on parameter values, positions DE favorably. Moreover, in comparison to Particle Swarm Optimization (PSO) and Genetic Algorithms (GA), DE often exhibits robust performance, particularly in scenarios where the objective function lacks smoothness or contains irregularities [21]. Thus, DE's ability to effectively navigate the specific challenges posed by FP-Growth's parameter tuning, coupled with its versatility, makes it a compelling choice over other optimization algorithms in this context.

The DE algorithm operates on a population of candidate solutions, perturbing and recombining them to explore the solution space effectively. Mathematically, the mutation and crossover operations for each candidate solution x_i at iteration $t + 1$ can be expressed as follows:

$$\mathbf{v}_i^{(t+1)} = \mathbf{x}_{r_1}^{(t)} + F \cdot (\mathbf{x}_{r_2}^{(t)} - \mathbf{x}_{r_3}^{(t)}) \quad (9)$$

$$\mathbf{u}_i^{(t+1)} = \begin{cases} \mathbf{v}_i^{(t+1)} & \text{if } \text{rand}() \leq \text{CR or } j = \text{rand}(D) \\ \mathbf{x}_i^{(t)} & \text{otherwise} \end{cases} \quad (10)$$

Here, $\mathbf{x}_{r_1}^{(t)}$, $\mathbf{x}_{r_2}^{(t)}$, and $\mathbf{x}_{r_3}^{(t)}$ are randomly selected solutions from the population, F is the scaling factor, D is the problem dimension, $\text{rand}()$ generates a random number in $[0, 1)$, and $\text{rand}(D)$ selects a random index in the range $[1, D]$.

The DE algorithm iteratively refines the candidate solutions, guiding the search toward optimal parameter values. Through iterative refinement, DE effectively navigates the solution space, leading to enhanced performance in the parameter-tuning process.

The performance metric utilized to optimize the parameters is based on maximizing accuracy while minimizing false positive rates. Thus, in each iteration of the algorithm, we calculate the value of *Accuracy*. Once the parameter tuning for a specific activity is completed, we run the algorithm one final time with the best parameters to extract the patterns of that activity. These extracted patterns form the core of the profile for each activity.

3.3. Neural network structure

This phase details the integration process for the components discussed earlier to construct a comprehensive profiling system. To summarize, we generate a minimum of one profile for each DNS activity. In cases where multiple optimal feature sets exist for certain activities, we create multiple profiles. These profiles are then integrated through a Neural Network (NN) structure, depicted in Fig. 1.

The initial layer of this network includes the selected features for each activity. These selected features which are for each DNS flow are transformed into the matrix form (where each row is the selected feature value) to facilitate processing and analysis within the neural network architecture. Subsequently, we normalize feature values into predefined ranges. Within this layer (the first hidden layer), the activity function of each node computes the corresponding feature range for each input value.

Moving forward, the subsequent layer focuses on pattern similarity assessment, encompassing all created profiles. Here, we calculate the similarity between the input pattern and all extracted patterns, determining the value for the most analogous pattern. The output from this layer, ranging between zero and one, signifies the degree of resemblance to the extracted patterns, with one indicating a precise match and zero implying no correspondence. Importantly, each function's output in this layer carries a weight signifying its differential impact on the final output. These weights encapsulate the varying influences of different created profiles within distinct activities, a topic we delve into further in subsequent sections.

Following this, the fourth layer consolidates the outputs of all created profiles per activity with their respective weights, leading to the ultimate decision. The output, a floating-point number, reflects the likelihood of the input's association with a particular activity. In this layer, the output is computed by summing the weighted product of profile outputs.

Concluding at the output layer, a softmax function, defined by the subsequent equation, is employed. This function utilizes the values from the fourth layer for diverse activities to select the associated profile based on their relative strengths:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} \quad (11)$$

where p_i represents the probability associated with the i th activity, z_i encapsulates the inherent value derived from the aggregation layer specific to that activity. The denominator's summation term ensures necessary normalization, resulting in a well-defined probability distribution. These calculated probabilities serve as a refined foundation for identifying and selecting the most appropriate activity corresponding to each input, embodying the profound decision-making mechanism of the proposed approach.

• Weight Calculation

The subsequent phase involves computing weights for each created profile output in each activity. This critical step incorporates an additional training process, combining backpropagation with a selected optimizer for iterative weight adjustments.

For optimization, we conducted tests with various optimizers and found that the Adagrad optimizer [22] demonstrates superior performance in this context. The objective is to minimize the Categorical Cross-Entropy loss function, suitable for multiclass classification scenarios:

$$\text{Categorical Cross-Entropy Loss} = - \sum_{i=1}^N y_i \log(p_i) \quad (12)$$

Here, N represents the number of classes, y_i signifies the actual target probability for class i , and p_i denotes the predicted probability.

Weight Calculation Procedure:

- **Initialization:** Initialize profile weights randomly, such as $w_p \sim \mathcal{N}(0, \sigma^2)$.
- **Forward Propagation:** For each activity a , the output is calculated based on the weighted sum of the final profile outputs:

$$\text{aggregation}_a = \sum_{p=1}^P w_p \cdot o_{p,a} \quad (13)$$

- **Activity Probability:** Apply the softmax activation function to obtain the probability of each activity:

$$p_a = \frac{e^{\text{aggregation}_a}}{\sum_{a'} e^{\text{aggregation}_{a'}}} \quad (14)$$

- **Loss Function:** Calculate the Categorical Cross-Entropy loss for all activities:

$$\text{loss} = - \sum_{a=1}^A y_a \log(p_a) \quad (15)$$

where y_a is the true target probability for activity a .

- **Backpropagation:** To calculate the gradient of the loss with respect to the profile weights w_p , we consider the chain rule and the gradients of the loss with respect to the intermediate values.

$$\frac{\partial \text{loss}}{\partial p_a} = - \frac{y_a}{p_a} \quad (16)$$

Next, compute the gradient of the activity probability with respect to the aggregation value aggregation_a :

$$\frac{\partial p_a}{\partial \text{aggregation}_a} = p_a(1 - p_a) \quad (17)$$

Finally, for a profile weight w_p , the gradient of the loss with respect to that weight can be calculated using the chain rule:

$$\begin{aligned} \frac{\partial \text{loss}}{\partial w_p} &= \sum_{a=1}^A \frac{\partial \text{loss}}{\partial p_a} \cdot \frac{\partial p_a}{\partial \text{aggregation}_a} \cdot \frac{\partial \text{aggregation}_a}{\partial w_p} \\ &= \sum_{a=1}^A \left(-\frac{y_a}{p_a} \right) \cdot (p_a(1 - p_a)) \cdot o_{p,a} \\ &= - \sum_{a=1}^A (y_a - p_a) \cdot o_{p,a} \end{aligned} \quad (18)$$

where $\frac{\partial \text{loss}}{\partial p_a}$ is the gradient of the loss with respect to the activity probability p_a . Similarly, $\frac{\partial p_a}{\partial \text{aggregation}_a}$ is the gradient of the activity probability with respect to the aggregation value aggregation_a . Lastly, $o_{p,a}$ is the output of profile p for activity a .

- **Adagrad Weight Update:** The updated profile weight w_p is obtained using the Adagrad optimizer:

$$w_p = w_p - \alpha \cdot \frac{g_p}{\sqrt{\sum_{t=1}^T (g_{tp})^2 + \epsilon}} \quad (19)$$

where α is the learning rate, g_p is the gradient of w_p , $(g_{tp})^2$ is the squared gradient of w_p at each time step t , T is the total number of time steps, and ϵ prevents division by zero.

- **Exponential Moving Averages:** The moving averages are updated during training iterations:

$$\hat{m}_p = \beta_1 \cdot \hat{m}_p + (1 - \beta_1) \cdot \frac{\partial \text{loss}}{\partial w_p} \quad (20)$$

$$\hat{v}_p = \beta_2 \cdot \hat{v}_p + (1 - \beta_2) \cdot \left(\frac{\partial \text{loss}}{\partial w_p} \right)^2 \quad (21)$$

where β_1 and β_2 are the exponential decay rates for the first and second moments, respectively.

- **Iteration:** Repeat the forward propagation, loss computation, backpropagation, Adagrad weight update, and exponential moving average update steps for a predetermined number of epochs.

The training process, leveraging backpropagation and the adaptive learning rate properties of the Adagrad optimizer, iteratively adjusts profile weights, contributing to overall loss reduction. This iterative process enhances the model's predictive capability, converging as the model learns to adjust profile weights and network parameters, achieving minimized loss and improved accuracy and predictive performance.

It is essential to clarify that the proposed neural network model is designed to be adaptable and applicable across various datasets within the domain of DNS-based malicious activity analysis. While certain adjustments may be necessary when transitioning to different datasets, the fundamental architecture and principles of the neural network remain consistent. For instance, in this study, the dataset comprises five primary classes. Thus, in the last hidden layer, we incorporate five nodes. However, if another dataset features eight classes, we must adjust the number of nodes in the final hidden layer accordingly. Similar modifications may be necessary in other layers, but the overarching concept, number of layers, and their connectivity remain consistent.

In terms of structure complexity, this model adopts a hybrid approach that lies between a basic fully connected neural network and more complex structures such as CNNs or LSTMs. Specifically, the network comprises fully connected layers, with each layer interconnected to facilitate information propagation. However, certain edges within the network may weigh zero, and backpropagation is performed only on the weights of the second hidden layer outputs (on the outputs of each profile to determine the impact of each created profile on the system output).

Furthermore, within this structure, each profile generated in the second hidden layer only takes into account a specific set of features. Consequently, the connections from those features (i.e., the nodes representing those features) are assigned a weight of

one, while all other features are assigned a weight of zero. This principle also applies to the first hidden layer, where we conduct range calculations for each feature within every label. Therefore, if there are n labels, the first hidden layer will contain at most n nodes for the range calculation of each feature. This arises from the distinct behavior exhibited by each feature across labels.

Thus, for any label with one or more profiles created in the second hidden layer utilizing a particular feature, denoted as f_i , during the pattern extraction phase, there will be a corresponding node, labeled as r_i , in the first hidden layer. The connection between r_i and f_i carries a weight of one, while all connections from r_i to other features f_j ($i \neq j$) carry a weight of zero. Furthermore, the input to node r_i is derived from both the input layer and the specific node representing feature f_i .

In summary, within the input layer, each node f_i is connected to at least one and at most n (where n represents the number of labels) output edges. Moving to the first hidden layer, the number of nodes equals the count of unique features utilized across all profiles generated within each label. For instance, consider a scenario with 5 labels. If each label has two profiles, each utilizing 4 distinct features, then there are 8 features per profile. Assuming no feature duplicates within these 8 features, the first hidden layer would consist of 8 nodes. Consequently, in this setup, there would be $8 \times 5 = 40$ nodes within the first hidden layer, while the input layer would contain at least 8 and at most 40 nodes.

3.4. Behavior similarity

This study introduces a novel metric, behavior similarity, to quantitatively measure the similarity between different DNS activities. The behavior similarity formula (Eq. (22)) leverages feature selection graphs for all activities, evaluating the similarity between activities A_1 and A_2 using the function $S(A_1, A_2)$. The calculation involves examining edges connecting features f_i and f_j in graphs E_1 and E_2 , updating similarity iteratively:

$$S(A_1, A_2) = \#correlations - \sum_{f_i, f_j}^{\text{Features}} |E_1 - E_2| \quad (22)$$

Initially set to the total number of correlations, the similarity decreases as the absolute difference in corresponding edges is calculated. The final similarity is normalized by dividing it by the total number of correlations, resulting in a value between -1 and $+1$.

The behavior similarity algorithm has broad applications, enabling precise quantification of diverse network activities. It proves valuable in anomaly detection, intrusion detection, profiling, and security threat identification. By identifying similar activities, security professionals can proactively address risks, enhancing network defenses. The algorithm also supports comprehensive comparisons against known behaviors, aiding in the accurate classification and categorization of network traffic. This enhances network analysis capabilities, empowering organizations with precision and informed decision-making.

4. Implementation

This section presents a comprehensive overview of ALFlowLyzer, the innovative application layer network flow analyzer package [23]. We describe its architecture, highlighting its unique features and advantages. Additionally, we delve into the flow creation process, encompassing network traffic capture and extracting relevant features for the profiling model. Furthermore, we discuss the critical aspects of behavior selection and feature extraction, which are pivotal in generating accurate profiles and labels.

4.1. ALFlowLyzer

ALFlowLyzer is a Python-based software tool designed for analyzing network PCAP files and generating analyzed data in CSV format. This tool parses PCAP files, extracts DNS flows, and identifies meaningful features from the application layer of packets. While ALFlowLyzer can create flows for various application layer protocols, this work focuses specifically on the DNS protocol. To use ALFlowLyzer, users need to provide a network PCAP file as input. The tool will parse the file, identify DNS flows, extract relevant features, and output this information into a CSV file. Users must then manually label each DNS flow in the CSV file. The labeled CSV file can subsequently be used to train ML or DL models for further analysis and detection tasks. For detailed instructions on using ALFlowLyzer and understanding its functionalities, please refer to the ALFlowLyzer GitHub repository.

ALFlowLyzer's primary strength lies in its versatility, capable of extracting features from various application layer protocols. Despite its adaptability, this study focuses specifically on the DNS protocol, allowing for in-depth analysis and profiling tailored to this protocol. ALFlowLyzer's specialized approach enhances its analytical capabilities, providing valuable insights for the accurate identification of malicious activities at the application layer. It remains flexible for future protocol inclusion, showcasing its applicability in diverse research endeavors.

ALFlowLyzer's feature extraction covers a wide range of application layer attributes, including DNS-specific features as depicted in Fig. 3, Tables 1, and 2. These features serve as the foundation for subsequent profiling and behavior analysis.

To ensure accuracy, ALFlowLyzer undergoes rigorous testing with diverse datasets and real-world network traffic traces. This validation process, based on established methodologies, ensures high precision and consistency in feature extraction, reinforcing its efficacy as an application layer network flow analyzer.

The output is a structured dataset containing comprehensive information about application layer flows, facilitating deeper insights into network behavior. ALFlowLyzer empowers researchers and practitioners to identify anomalies and develop robust security mechanisms with its rich features specific to the application layer.

In summary, ALFlowLyzer is a powerful and versatile application layer flow analysis tool focusing on enhanced accuracy and specificity in profiling application layer behavior. Subsequent subsections provide a detailed overview of the rich features and functionality offered by ALFlowLyzer.

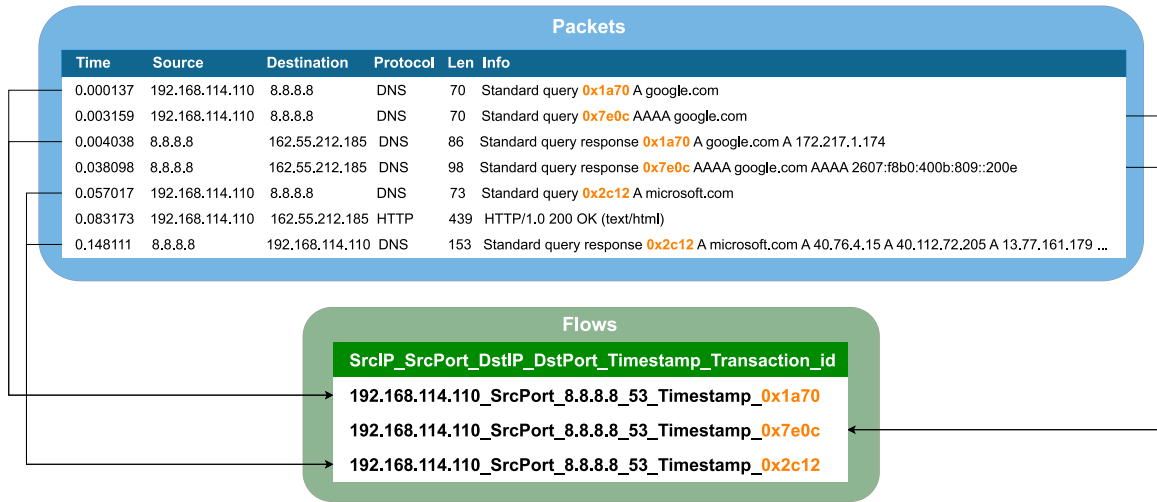


Fig. 2. Flow creation.

Table 1
ALFlowLyzr metadata features.

Feature category	Feature	Feature name	Feature	Feature name
DNS lexical	F1	Domain Name	F7	Domain Name 2-Gram
	F2	Top Level Domain	F8	Domain Name 3-Gram
	F3	Second Level Domain	F9	Numerical Percentage
	F4	Domain Name Length	F10	Character Distribution
	F5	Sub Domain Name Length	F11	Character Entropy
	F6	Domain Name 1-Gram		
DNS statistical	F12	Continuous Numeric Max Len	F20	TTL Values Max
	F13	Continuous Alphabet Max Len	F21	TTL Values Mean
	F14	Continuous Consonant Max Len	F22	TTL Values Mode
	F15	Continuous Same Alphabet Max Len	F23	TTL Values Var
	F16	Vowel Consonant Ratio	F24	TTL Values Std
	F17	Conv Freq Vowel Consonant	F25	TTL Values median
	F18	Distinct TTL Values	F26	TTL Values Skew
	F19	TTL Values Min	F27	TTL Values CoV
DNS resource record-based	F28	Distinct A Resource Records	F33	Query Resource Record Type
	F29	Distinct NS Resource Records	F34	Answer Resource Record Type
	F30	Average Authority Resource Records	F35	Query Resource Record Class
	F31	Average Additional Resource Records	F36	Answer Resource Record Class
DNS third-party	F32	Average Answer Resource Records		
	F37	WHOIS Domain Name	F45	Domain Organization
	F38	Domain Email	F46	Domain Address
	F39	Domain Registrar	F47	Domain City
	F40	Domain Creation Date	F48	Domain State
	F41	Domain Expiration Date	F49	Domain Zipcode
	F42	Domain Age	F50	Domain Name Servers
	F43	Domain Country	F51	Domain Updated Date
	F44	Domain DNSSEC		

4.2. Flow creation

Our approach to creating network flows diverges from previous works. We recognize the limitations of focusing solely on the UDP level and emphasize the need for a comprehensive consideration of both network and application layers. Acknowledging the complexity of protocols within the application layer, we adopt a protocol-specific flow definition approach for accurate representations of application layer behaviors.

Traditionally, network flow analysis concentrated on the network layer, neglecting application layer intricacies. In response, we adopt a holistic perspective and acknowledge the impracticality of a unified flow definition across all application layer protocols. Instead, we define flows on a protocol-by-protocol basis, ensuring precision in representing diverse application layer behaviors.

Unlike traditional time-based flow definitions, the proposed approach identifies the DNS header's transaction ID as the primary identifier for DNS flows. This transaction ID, a reliable means of differentiation and analysis, coupled with fundamental network layer attributes, facilitates temporal differentiation and detection of anomalous behaviors like Poisoning Attacks [24]. Fig. 2 visually illustrates the systematic flow-creation process.

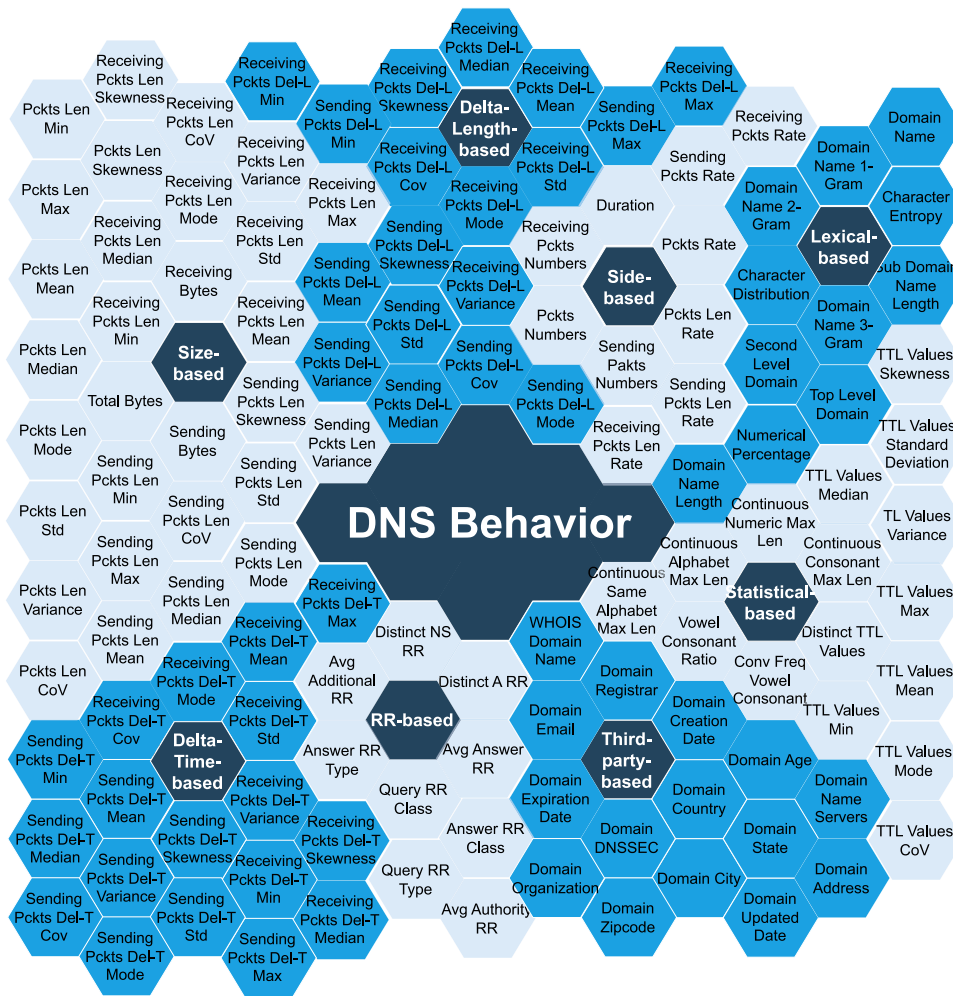


Fig. 3. DNS behavior.

Flow termination in this approach relies on two criteria: exceeding a predetermined maximum flow duration and surpassing the maximum flow idle time. These criteria ensure an accurate representation of network traffic dynamics, capturing diverse communication patterns and providing valuable insights into network behavior. Including a timestamp attribute in flow definition and adopting multiple closure conditions distinguish this methodology, enhancing its significance in network flow analysis.

In summary, the DNS flow creation method efficiently captures diverse communication patterns within the application layer, addressing the limitations of previous approaches. By embracing application layer intricacies and refining flow definition and termination criteria, this approach advances application layer flow analysis techniques, enhancing the accuracy of network traffic profiling for improved security and performance optimization.

4.3. Behavior selection and feature extraction

An effective network behavior analysis model hinges on meticulous behavior selection and feature extraction. This approach, illustrated in Fig. 3, identifies categories such as DNS Lexical-based, DNS Statistical-based, DNS Resource Record-based, DNS Third-party-based, Size-based, Delta-length-based, Delta-time-based, and side-based behaviors. The last category, side behavior, encompasses behaviors not fitting into the mentioned classifications. The selection of these categories results from a comprehensive literature review and thorough analysis of network traffic patterns.

To delve into these behaviors, we have developed ALFlowLyzer, a robust application layer feature extractor capable of analyzing network traffic and extracting 130 features from both meta-data and flow statistical data sources [3,4].

Flow statistical features encompass measures derived from network flow characteristics, including duration, packet count, byte count, inter-arrival time, and payload size distribution. These features unveil statistical patterns in network flows, aiding in anomaly identification and traffic classification. Conversely, meta-data features extract information directly from application layer protocols,

Malicious DNS Behavior Profiling through Application Layer Traffic Analysis

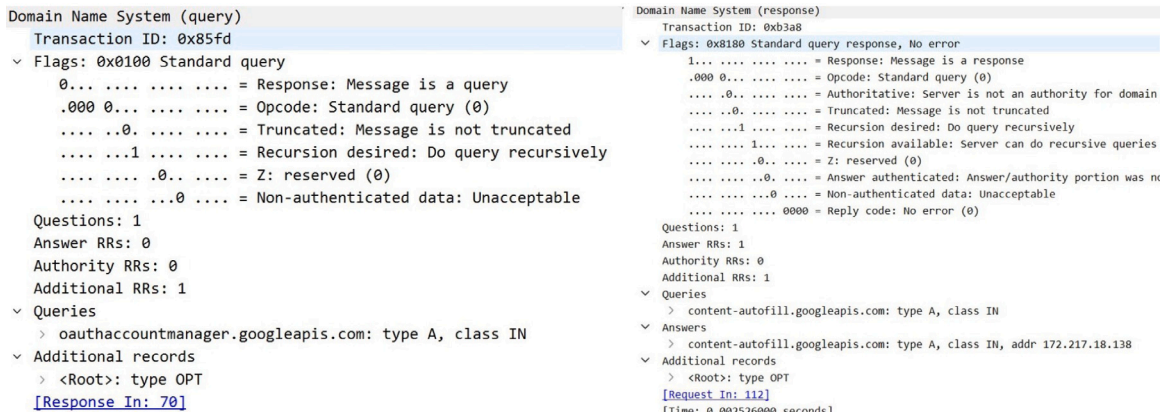


Fig. 4. DNS metadata captured from Wireshark.

providing deeper insights into specific behaviors and communication patterns. The fusion of statistical flow features and meta-data features in the extraction process facilitates a holistic network behavior analysis, enhancing model accuracy and effectiveness.

For detailed feature definitions, readers are encouraged to consult the ALFlowLyzer project's GitHub repository [23].

In summary, the behavior selection and feature extraction approach leverage distinct network behavior categories. ALFlowLyzer's development empowers us to extract a rich set of features, enabling comprehensive network behavior analysis and accurate profiling, classification, and anomaly detection.

4.3.1. Flow statistical features

Flow statistical features are obtained by employing statistical functions such as min, max, mean, mode, variance, standard deviation, median, skewness, and coefficient of variation to the network traffic flow. For each flow, the consecutive receiving/sending packet lengths and the time intervals between each receiving/sending consecutive packet are compared to obtain these features. These features can be classified into two main categories - (1) Packet Length-based Features and (2) Packet Delta Time-based Features. The ALFlowLyzer extracts 79 features from network traffic flow statistics.

4.3.2. Meta data features

Meta-data refers to the additional information present in the application layer of the traffic flow. It consists of data that is specific to a particular protocol. We provide an example by referring to Fig. 4, which depicts a snap taken from Wireshark. It represents the DNS protocol's meta-data for query and response messages. The ALFlowLyzer extracts 51 features from the meta-data of the network traffic flow. The 51 meta-data features have been summarized in Table 1.

DNS Lexical Features: Attackers generate numerous random domain names on the run to evade detection. The domain-generating algorithm is responsible for generating such arbitrary domain names as malicious domains. Lexical features related to the language's words or vocabulary are extracted from these domain names. The extracted lexical features are domain name, top-level domain, second-level domain, domain name length, sub-domain name length, 1-g, 2-g, and 3-g of the domain name, numerical percentage, character distribution, and character entropy. The lexical features of such malicious domains significantly differ from their benign counterparts.

DNS Statistical Features DNS statistical features refer to the information extracted from the answer resource records using statistical analysis. The statistical functions include min, max, mean, mode, variance, standard deviation, median, skewness, and coefficient of variation.

- **Domain Name-based Features:** These features are derived by employing statistical functions for characters, numbers, vowels, and consonants that form the domain name. The features that fall in this category are the maximum length of continuous numeric characters, the maximum length of continuous alphabetic characters, the maximum length of continuous consonants, the maximum length of continuous same alphabetic character, the ratio of vowels to consonants, and the conversion frequency of vowels to consonants.
- **TTL Value-based Features:** Time To Live (TTL) refers to the time limit that is set for the resolved DNS record. It indicates the amount of time which the authoritative name server can hold the resolved DNS record in its cache. These features are obtained by employing the statistical operations to the extracted TTL values from the DNS resource records.

DNS Resource Record-based Features There are four main categories of resource records in a DNS packet - (1) Query resource records, (2) Answer resource records, (3) Authority resource records, and (4) Additional resource records. DNS resource record-based features are extracted from the information present in these resource records. The extracted features in this category include distinct

Table 2
ALFlowLyzer common features.

Feature category	Feature	Feature name	Feature	Feature name
Size-based	F52	Total Bytes	F67	Sending Packets Len median
	F53	Sending Bytes	F68	Sending Packets Len Mode
	F54	Receiving Bytes	F69	Sending Packets Len Std
	F55	Packets Len Min	F70	Sending Packets Len Var
	F56	Packets Len Max	F71	Sending Packets Len CoV
	F57	Packets Len Mean	F72	Sending Packets Len Skew
	F58	Packets Len median	F73	Receiving Packets Len Min
	F59	Packets Len Mode	F74	Receiving Packets Len Max
	F60	Packets Len Std	F75	Receiving Packets Len Mean
	F61	Packets Len Var	F76	Receiving Packets Len median
	F62	Packets Len CoV	F77	Receiving Packets Len Mode
	F63	Packets Len Skew	F78	Receiving Packets Len Std
	F64	Sending Packets Len Min	F79	Receiving Packets Len Var
	F65	Sending Packets Len Max	F80	Receiving Packets Len CoV
	F66	Sending Packets Len Mean	F81	Receiving Packets Len Skew
Delta-Length-based	F82	Sending Packets Delta Len Min	F91	Receiving Packets Delta Len Min
	F83	Sending Packets Delta Len Max	F92	Receiving Packets Delta Len Max
	F84	Sending Packets Delta Len Mean	F93	Receiving Packets Delta Len Mean
	F85	Sending Packets Delta Len median	F94	Receiving Packets Delta Len median
	F86	Sending Packets Delta Len Std	F95	Receiving Packets Delta Len Std
	F87	Sending Packets Delta Len Var	F96	Receiving Packets Delta Len Var
	F88	Sending Packets Delta Len Mode	F97	Receiving Packets Delta Len Mode
	F89	Sending Packets Delta Len CoV	F98	Receiving Packets Delta Len CoV
	F90	Sending Packets Delta Len Skew	F99	Receiving Packets Delta Len Skew
Delta-Time-based	F100	Sending Packets Delta Time Min	F109	Receiving Packets Delta Time Min
	F101	Sending Packets Delta Time Max	F110	Receiving Packets Delta Time Max
	F102	Sending Packets Delta Time Mean	F111	Receiving Packets Delta Time Mean
	F103	Sending Packets Delta Time median	F112	Receiving Packets Delta Time median
	F104	Sending Packets Delta Time Std	F113	Receiving Packets Delta Time Std
	F105	Sending Packets Delta Time Var	F114	Receiving Packets Delta Time Var
	F106	Sending Packets Delta TimeMode	F115	Receiving Packets Delta Time Mode
	F107	Sending Packets Delta Time CoV	F116	Receiving Packets Delta Time CoV
	F108	Sending Packets Delta Time Skew	F117	Receiving Packets Delta Time Skew
Side-based	F118	Duration	F123	Receiving Packets Rate
	F119	Packets Numbers	F124	Sending Packets Rate
	F120	Receiving Packets Numbers	F125	Packets Len Rate
	F121	Sending Packets Numbers	F126	Receiving Len Packets Rate
	F122	Packets Rate	F127	Sending Len Packets Rate

address resource records, distinct name server resource records, the average number of authority, additional and answer resource records, the type of answer and query resource records, and the class of answer and query resource records.

DNS Third-Party Features DNS third-party features are derived from third-party sources, such as WHOIS. WHOIS is a response and query protocol that stores information regarding registered users of an internet resource, such as a domain name or IP address block. So, these features mostly contain the domain's biographical properties. The extracted features in this category are WHOIS domain name, email, registrar, creation date, expiration date, age, country, DNSSEC, organization, address, city, state, zip code, name servers, and updated date.

This section provided a thorough overview of ALFlowLyzer, the innovative application-layer network flow analyzer. It highlighted unique features, advantages, and the underlying architecture. The detailed discussion covered flow creation, behavior selection, and feature extraction, emphasizing a meticulous approach to capturing and analyzing application-layer flows. ALFlowLyzer equips researchers and practitioners with a potent tool for understanding network behaviors, anomaly detection, and refining security measures.

5. New malicious DNS dataset (BCCC-CIC-Bell-DNS-2024)

This section begins by examining existing publicly available datasets within the domain, followed by a rationale for integrating the chosen datasets. Additionally, we present comprehensive details about the recently introduced dataset, BCCC-CIC-Bell-DNS-2024, achieved through the integration of prior datasets using ALFlowLyzer.

5.1. Available DNS datasets

In recent years, the following represent prominent DNS-related datasets:

Boss of the SOC Dataset Version 1 (Botsv1): This publicly available dataset focuses on DNS-based Command and Control (C&C) traffic. It includes evidence from real security events or lab recreations, divided into original and attack-only versions. The original dataset, accessible in various formats, contains 6.1 GB of compressed JSON and CSV files, organized by source type [25].

PUF Dataset (ICCIDIS 2018): This dataset comprises flows from Panjab University's Computer Centre, with 260,343 benign and 38,120 abnormal flows. It aids in detecting suspicious sub-networks based on statistically determined and entropy-based attributes [26].

Labeled FQDN/IP dataset (Computers & Security): This dataset uses FQDN/IP blacklists, whitelists, and domain reputation tools to label traffic for training and evaluation. It incorporates seven blacklists, top Alexa sites, and domain-based blacklists from Torpig and Conficker botnets [27].

CIC-Bell-DNS-2021 (University of New Brunswick): With 400,000 benign and 13,011 malicious samples, this dataset categorizes domains into malware, spam, phishing, and benign categories. It includes 32 distinct features, covering lexical, statistical, and biographical aspects [3].

CIC-Bell-DNS-EXF-2021 (University of New Brunswick): This dataset, comprising 270.8 MB of DNS traffic, includes benign, light attack, and heavy attack samples. Extracting 30 characteristics using a proposed feature extractor results in a structured dataset with 323,698 heavy attack samples, 53,978 light attack samples, and 641,642 unique benign samples. All traffic is recorded, and timestamps are classified using tcpdump [4].

5.2. Integration of selected datasets

The careful selection of CIC-Bell-DNS-2021 and CIC-Bell-DNS-EXF-2021 datasets was driven by a strategic consideration of the limitations observed in commonly employed public DNS datasets. This choice is underpinned by the imperative need for datasets that encapsulate a spectrum of network traffic scenarios, thereby fortifying the robustness of the model evaluation.

The rationale for integrating CIC-Bell-DNS-2021 and CIC-Bell-DNS-EXF-2021 is multi-faceted. These datasets present a snapshot of real-world network traffic, encompassing a rich spectrum of benign and malicious DNS activities. By incorporating these datasets into the analytical framework through ALFlowLyzer, we aim to address the constraints of conventional datasets and enhance the sophistication of the model evaluation.

The imperative for integration stems from the unique characteristics offered by the CIC-Bell-DNS-2021 and CIC-Bell-DNS-EXF-2021 datasets. Their inclusion diversifies the analysis pipeline, affording a nuanced understanding of network behavior. This diversity is pivotal for comprehensively evaluating the profiling model, as it allows us to navigate intricate network dynamics and realistically evaluate its capabilities.

Furthermore, the uniform architecture of both datasets significantly streamlines the integration process, facilitating seamless compatibility and synergy between the two. With similar structures, the integration endeavor is marked by reduced complexities and fewer compatibility issues, allowing for a more straightforward and effective integration between the datasets.

The steps for integrating those two datasets are as follows:

- **Cleaning:** Initially, we examined the PCAP files of both datasets, cross-referencing the information provided on their respective websites and papers. During this phase, any PCAP files whose information did not align with their websites' content were eliminated. Additionally, packets not conforming to the correct format were removed from consideration.
- **Improvement:** In this phase, the PCAP files are inputted into ALFlowLyzer. The resulting CSVs from ALFlowLyzer exhibit two primary distinctions from the original datasets' CSV files. Firstly, each row now represents a DNS flow, whereas the original datasets followed a packet-based approach per domain name. Secondly, the CSVs now contain additional features (columns), and implementing the original datasets' features has been fixed and standardized.
- **Labeling:** Labeling Process: The final stage encompasses the meticulous labeling of the CSV rows. Given the transition to a new CSV format (flow-based rather than packet-based), the initial labeling relied on information provided on the datasets' web pages and in related papers. As a part of this process, we cross-referenced the information for each row (DNS flow) with the labeling information from the original dataset and assigned labels accordingly. Subsequently, we employed the defined behavior similarity method on the resulting data and thoroughly reviewed the labeled data. As a result of this secondary labeling procedure, we decided to merge different types of DNS Exfiltration activities into a single general label called "Exfiltration".

The enhanced dataset boasts significant improvements across various dimensions. These include an expanded range of labels, improved data labeling processes, thorough data cleaning procedures, and a refined data format. Notably, in the new dataset, each row represents a DNS flow, departing from the domain-centric approach. Additionally, it is worth highlighting that our dataset boasts over 120 features, surpassing the previous datasets which contained less than 60 features each.

Additional information regarding the integration steps of these two datasets and details about the new dataset are accessible on the dataset webpage.

Table 3
Flows number per activity in the BCCC-CIC-Bell-DNS-2024.

Activity	Flows number	Activity	Flows number
Benign	3,545,212	Text	24,515
Malware	81,698	Image	31,628
Spam	30,371	Video	37,106
Phishing	43,348	Compressed	33,854
Audio	33,114	Exe	36,523

Table 4
Selected feature for each activity in each profile.

Activity	1st profile	2nd profile
Benign	{F53, F64, F60, F58}	{F53, F57, F56, F67}
Malware	{F121, F63, F88, F85}	{F121, F63, F84, F84}
Spam	{F60, F58, F84, F92}	{F82, F94, F53, F62}
Phishing	{F66, F67, F65, F64}	{F57, F94, F61, F56}
Exfiltrate	{F125, F53, F4, F17}	{F12, F11, F74, F66}

5.3. New dataset structure

Flow extraction and CSV generation using ALFlowLyzer have played a pivotal role in dataset preparation for analysis. ALFlowLyzer's capabilities enabled the extraction of essential flows from raw network traffic data, generating corresponding CSV files for subsequent processing and analysis.

Table 3 presents detailed information on the new dataset's number of flows per label. Notably, the CIC-Bell-DNS-EXF-2021 dataset [4] furnishes insights into DNS data Exfiltration attack traffic, categorizing attacks into two classes: light file attacks and heavy file attacks. Within each category, encompassing audio, compressed, exe, image, text, and video file types, six distinct file types were identified. This study consolidates light and heavy data, resulting in six unique sub-categories within the Exfiltration category. This integration facilitates a comprehensive examination of the distribution of various flow labels, thereby enhancing the comprehension of the dataset's structure and equilibrium.

Furthermore, regarding the features, Tables 1 and 2 provide a comprehensive overview of the extracted features. These features are divided into two primary categories, DNS Metadata features and common application layer features, as elucidated in the preceding section. Additionally, regarding data organization, we have adhered to the DNS flow definition, ensuring that each row in the new dataset represents a distinct DNS flow.

In conclusion, integrating "CIC-Bell-DNS-2021" and "CIC-Bell-DNS-EXF-2021" datasets into the newly created "BCCC-CIC-Bell-DNS-2024" dataset significantly enriches the dataset's richness and diversity for evaluating the proposed profiling model. Leveraging ALFlowLyzer on raw PCAP files from these datasets facilitated the generation of new CSV files containing extracted features and relevant information for comprehensive analysis. The subsequent section will utilize "BCCC-CIC-Bell-DNS-2024" to assess the efficacy of the proposed profiling model.

6. Experiment results

This section presents experiment results obtained from applying the new dataset to the proposed model. The experimental procedure encompasses feature selection, behavior similarity analysis, and training/testing of the profiling system.

6.1. Feature selection

We independently applied the feature selection algorithm to identify the best features for each DNS activity. Table 4 presents the selected features for each activity and profile. Two distinct profiles were constructed for each activity, each with four features. Overlapping features between profiles were minimized to enhance distinctiveness. Fig. 5 visually illustrates feature correlations across different activities.

Additionally, edges with weights below 0.3 were removed to retain only the most relevant features, enhancing the efficiency and accuracy of subsequent analyses.

6.1.1. Third-party features

Including third-party features, precisely WHOIS information related to each domain, introduced an additional challenge during dataset preparation. We observed many null values associated with these features, primarily due to the absence of information for certain domains (domains' inconsistency), particularly those associated with malicious activities. To maintain the integrity of the dataset and avoid potential biases, we opted to exclude these features from our analysis. Removing third-party features with null values ensures a consistent and reliable dataset for subsequent analysis and profiling.

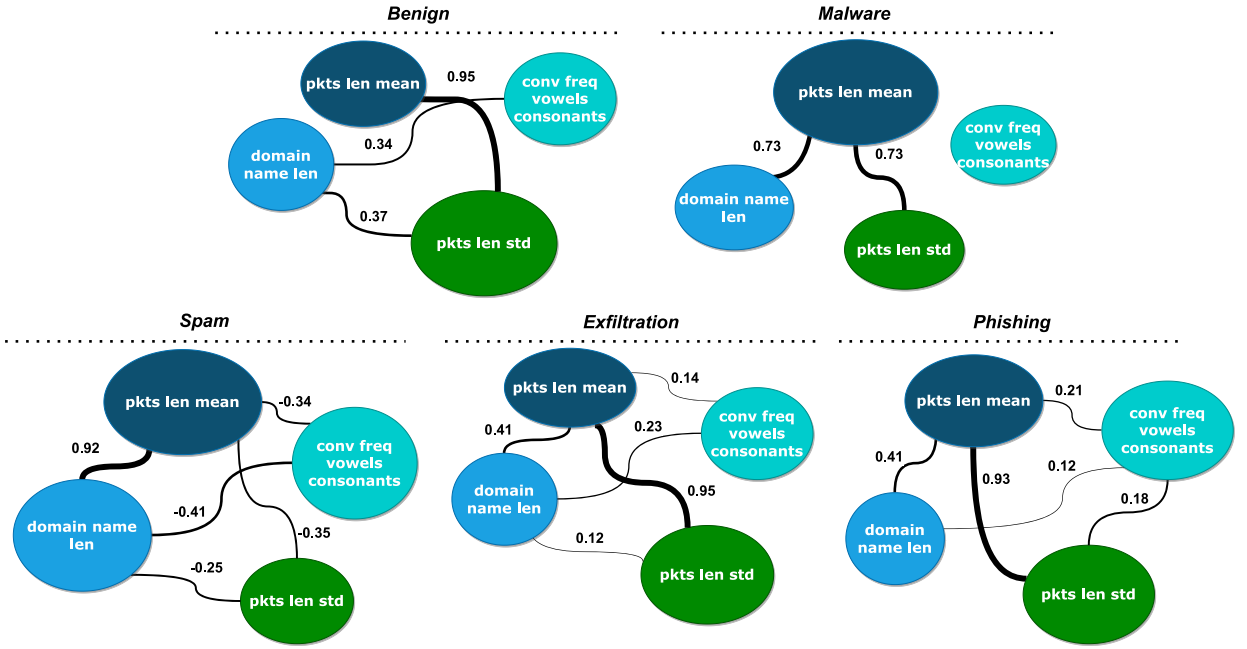
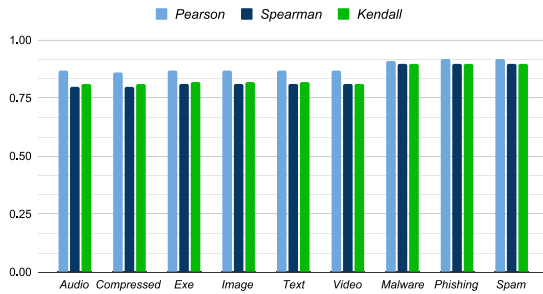


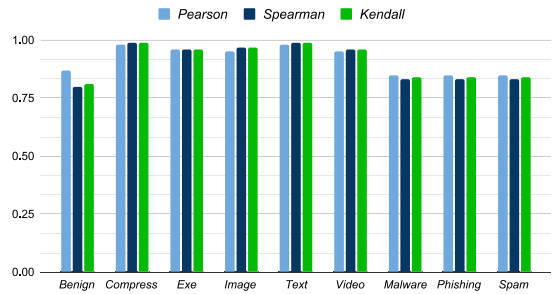
Fig. 5. Features correlations in different activities.

Similarity to Benign



(a) Behavior Similarity to Benign

Similarity to Audio



(b) Behavior Similarity to Audio-based Exfiltration

Fig. 6. Behavior similarity between different activities using different correlation calculation algorithms.

6.2. Behavior similarity

We employed the proposed behavior similarity algorithm to assess the degree of similarity between different network activities. The algorithm leveraged three correlation techniques: Pearson, Spearman, and KendallTau. Notably, while the algorithm quantified the similarity among all activities, this study focuses on presenting the similarity of each activity to exemplar activities, namely Benign and Audio. Fig. 6 showcases these specific comparisons visually.

6.2.1. High similarity of malicious activities

A notable observation emerged in the analysis of the CIC-Bell-DNS-EXF-2021 dataset—the striking similarity among malicious activities, as illustrated in Fig. 6. This finding prompted us to examine the malicious activities, meticulously examining their behavior feature-by-feature. As shown in Fig. 7, the analysis revealed high similarity among activities such as audio, video, image, executable, compressed, and text. To address this similarity and enhance the distinctiveness of the dataset, we consolidated these activities into a more general category labeled “Exfiltration”. This consolidation enabled us to capture the shared characteristics of these activities while ensuring their differentiation from other malicious behaviors.

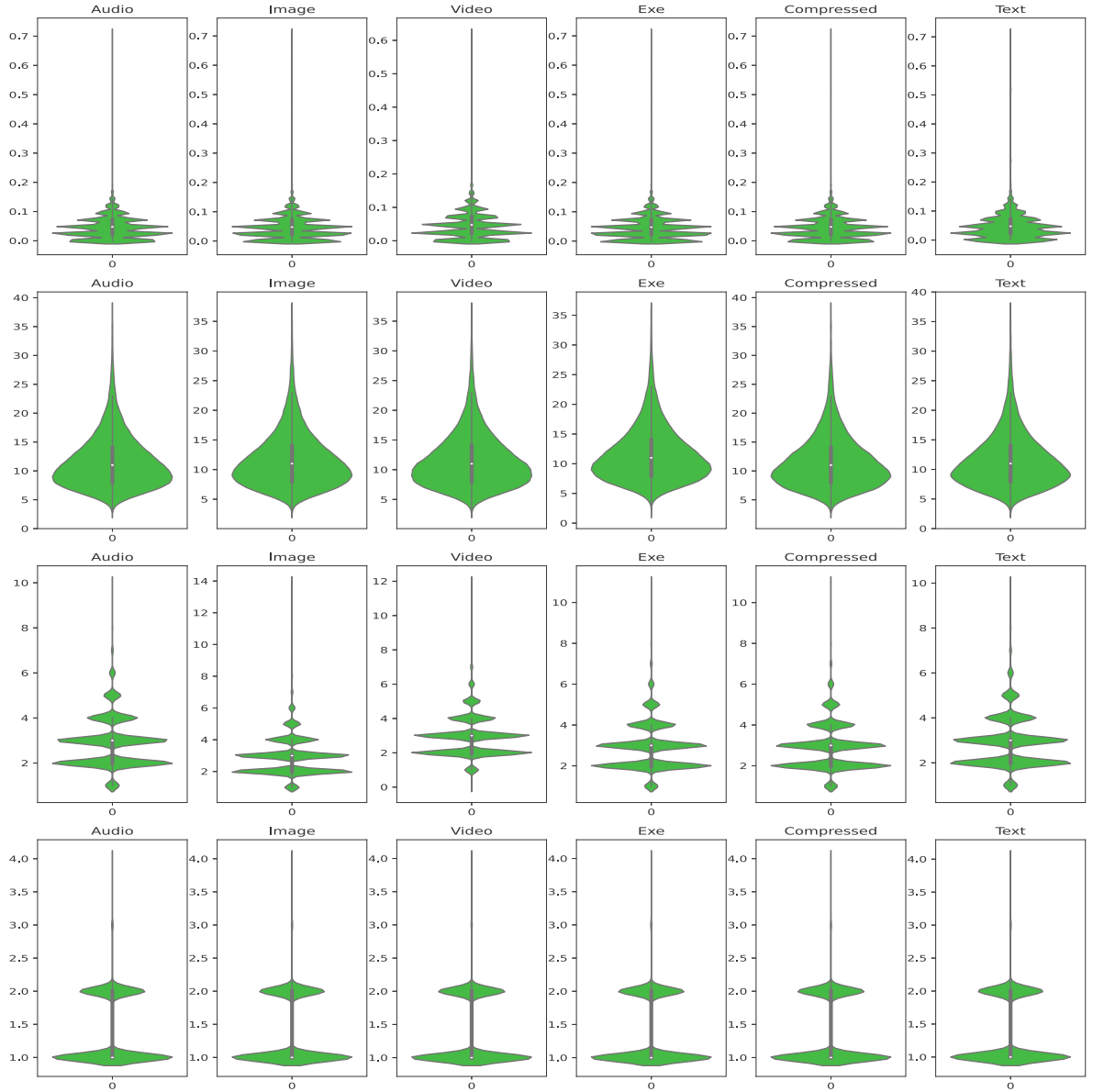


Fig. 7. Violin plot of `conv_freq_vowels_consonants`, `max_continuous_alphabet_len`, `max_continuous_consonants_len`, and `max_continuous_same_alphabet_len` features' values in different DNS Exfiltration activities.

6.3. Feature range calculation

The calculation of feature ranges was performed to prepare the features for subsequent pattern extraction steps. We determined the feature ranges for each activity by finding the main data point regions. The resulting violin plots depicted in Fig. 8 illustrate the variability of ranges among different activities, even for the same feature.

6.4. Profile creation

In this pivotal stage, we created profiles using selected features represented in Table 4 for each activity. Illustrative examples of the created profiles for select activities are depicted in Fig. 9, showcasing a profile comprising four features. Each node within the graph corresponds to a specific feature range, while edges symbolize the connections determined by the extracted rules.

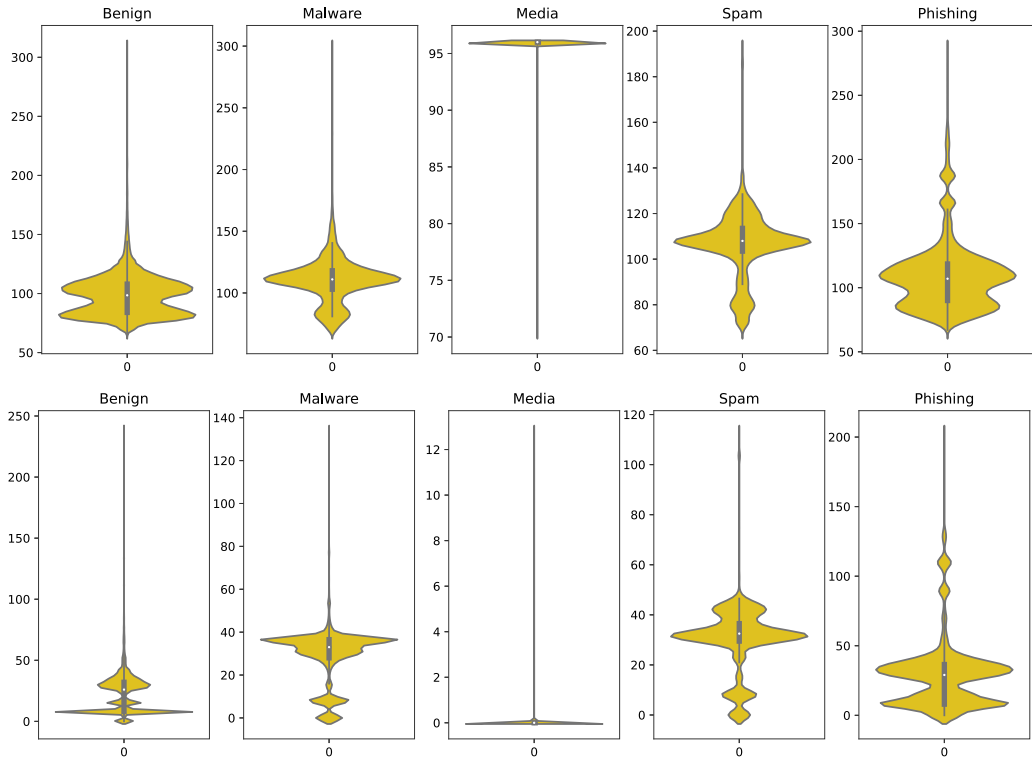


Fig. 8. Violin plot of mean_packets_len and standard_deviation_packets_len values in different activities (Media is another term we use for this dataset Exfiltration activities).

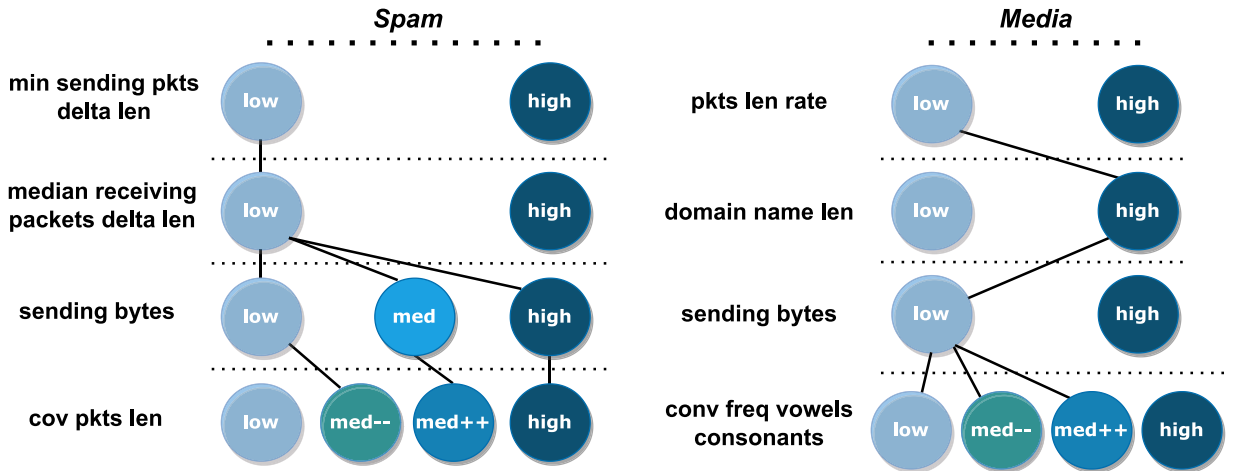


Fig. 9. Created profiles for some activities.

6.5. Performance

To evaluate the performance of the proposed model, we conducted extensive testing. Detailed evaluation results are in [Table 5](#).

7. Analysis and discussion

This section presents a concise analysis of our innovative behavior profiling model. We introduce the concept of DNS Flow to transcend traditional boundaries, providing unique insights into application layer activities. The study focuses on distinct behaviors exhibited by individual features and explores correlations, shedding light on diverse attack scenarios.

Table 5
Performance metrics for each activity.

Activity	Precision	Recall	F1-score
Benign	95.5	98.1	96.7
Malware	100	97.0	98.4
Spam	99.3	99.0	99.1
Phishing	96.1	99.1	97.5
Exfiltration	99.7	98.1	98.8

7.1. Analysis of key ideas

The proposed model relies on two key concepts. Firstly, each feature displays unique behavior across activities, exemplified by substantial variability in mean_packets_len (Fig. 8). Secondly, correlations between features differ across activities, illustrated by the domain_name_len and numerical_percentage example. Experimental results (Figs. 8 and 5) validate these concepts, forming the basis for a comprehensive profile construction.

Profile creation is initiated by identifying feature values for each activity. Correlations between transformed values are computed using Association Rule mining, revealing consistent co-occurrence patterns. Our profiling system integrates unique feature behavior and variable correlations, constructing a rational and comprehensive profile representing each activity.

7.2. Profile creation analysis

Profile creation involves meticulous feature selection to represent activity behavior accurately. This approach emphasizes feature ranges and correlations as foundational elements for defining activity characteristics.

Feature ranges are crucial in rule extraction, providing boundaries for each activity. These ranges serve as reference points, enabling the identification of patterns and variations specific to each activity and contributing to the overall effectiveness of the profile.

Similarly, Correlations between features play a pivotal role in rule extraction and feature selection. The proposed model leverages meaningful correlations to gain insights into interdependencies among different features within each activity. This ensures the construction of robust and accurate profiles that capture nuanced DNS traffic behavior.

Careful feature selection is paramount for effective profile creation. This approach employs a novel algorithm that prioritizes highly correlated features. Rigorous experimentation confirms that selecting non-correlated features yields insignificant rules, compromising profile accuracy and reliability.

7.3. Feature selection analysis

Constructing a robust profile relies on critical feature selection. The proposed methodology introduces a novel approach that emphasizes identifying highly correlated features. Through experimentation, we established that non-correlated features yield insignificant rules, affirming the importance and effectiveness of the feature selection algorithm.

The algorithm prioritizes features with significant correlations, contributing to the overall characterization of each activity. This allows us to highlight features that substantially impact DNS traffic behavior, unveiling intricate patterns and dependencies within the dataset. By selecting features with strong correlations, the profile creation captures the distinctive behavior of each activity.

7.4. Analysis of behavioral similarities

Analyzing behavioral similarities (Fig. 6) provides insightful contributions to behavior analysis and profiling. These findings validate the efficacy of feature selection and correlation calculation techniques while offering valuable implications for future work.

Discovering that activities most similar to Benign are Spam, Phishing, and Malware establishes a strong foundation for the credibility of this approach. The shared structure among activities like Audio, Image, Video, Compressed, Exe, and Text underscores the effectiveness of the proposed methodology, affirming the ability to detect and measure behavioral similarities across diverse data types.

The experimental findings have practical implications, potentially revolutionizing detection algorithms. Leveraging observed similarities enhances identifying and classifying malicious activities, improving response accuracy and timeliness. Despite minor variations in similarity values for different activities, the choice of correlation algorithm does not significantly impact accurately identifying behavioral similarities.

Nonetheless, challenges arise in identifying unique correlations for Audio compared to other Exfiltration-related activities. This suggests further research to optimize results for highly similar activities. In summary, the work exemplifies the potential of behavior analysis and profiling to enhance security, positioning the research at the forefront of advancing the field. Continuous refinement and understanding of behavioral similarities fortify system and network security in the digital landscape.

Table 6
Comparison with previous works.

Authors	Method	Accuracy	Class
[3]	KNN	97%	Multi
[4]	MLP	95%	Multi
[28]	Deep NN	95%	Multi
[29]	CNN	96%	Binary
[30]	ELM	96%	Binary
[11]	LSTM	94%	Multi
[12]	RF+CNN	97%	Multi
Proposed model		99%	Multi

7.5. Analysis of created profiles

The experiments on the proposed model yield remarkable insights, significantly advancing behavior analysis and profiling. Thorough analysis successfully profiles the behavior of each malicious activity, validating the system's capabilities in detecting distinctive attack patterns.

To illustrate the effectiveness, consider the Exfiltration activity profile (Fig. 9). The analysis emphasizes incorporating additional common features, such as size-based or time-based features, alongside DNS-related features for effective behavior profiling. These auxiliary features enrich the accuracy and depth of the analysis, providing a comprehensive understanding of DNS activities' intricate dynamics.

The Exfiltration activity profile reveals a notable characteristic—the high domain name length- that aligns with common practices in Exfiltration-related activities. This strengthens the profiling system's robustness, enabling accurate recognition of Exfiltration behavior based on domain characteristics.

Additionally, the key feature “sending_bytes” exhibits varying values in displayed profiles, offering valuable insights for behavior classification. These distinct values inform conclusions for new test inputs, aiding behavior categorization. Similar analyses for all created profiles consistently demonstrate the system's effectiveness in identifying unique patterns associated with each attack type.

7.6. Zero-day attack profiling

The pioneering profiling system effectively characterizes zero-day attacks through network traffic analysis. When encountering a new attack, distinct scenarios emerge:

- **Distinct Behavior:** If no existing profiles align with the new activity, it is provisionally labeled “Unknown” or “Zero-day Attack” until sufficient data permits dedicated profile creation.
- **Profile Overlap:** If the new activity shares elements with multiple profiles, refining detection involves eliminating shared patterns. Continued alignment implies malicious intent, warranting the “Unknown” or “Zero-day Attack” label until a specific profile is established.

This addresses the challenges of zero-day attacks by establishing a baseline of normal behavior and creating profiles for known malicious behaviors. These profiles serve as reference points for identified attacks, but the system excels in recognizing previously unseen attacks or variants.

Identifying and responding to novel threats defines the method as a zero-day attack profiling system. While engaging in abnormal activity profiling, the emphasis on rapidly classifying and mitigating activities exploiting unknown vulnerabilities sets this approach apart, fortifying network security against emerging threats.

7.7. Comparative analysis with prior works

To evaluate the novelty and effectiveness of the proposed model, we conduct a comparative analysis with prior works [3,4,11,12,28–30]. The first two works utilized the same datasets as this study, while the others employed different datasets but utilized DL methods. By benchmarking against existing research, insights into the advancements and contributions offered by this approach are gained, covering main results, accuracy, label diversity, and focus. The comparison summary can be found in Table 6.

Firstly, the proposed model exhibits superior performance in behavior profiling compared to prior works. Enhanced precision in identifying and categorizing different activities within DNS traffic is evident through extensive experimentation and evaluation.

This work stands out for its unique approach of integrating two distinct datasets, resulting in a significantly larger number of labels. This novel method enhances the diversity and comprehensiveness of covered activities, providing a more nuanced understanding of behaviors within DNS traffic.

Moreover, this work proposed model pioneers behavior profiles for individual activities, a step beyond prior works that focused on specific aspects of network security without explicit profiling. The introduction of DNS Flow, revolutionizing the understanding and profiling of protocols at the application layer, is a groundbreaking contribution that distinguishes this research.

The model presented in this work surpasses previous works in terms of accuracy. It achieves this by leveraging distinctive behavior for individual features and varying correlations between features across different activities. The utilization of association

rule mining techniques further enhances accuracy by capturing frequent co-occurrence patterns among feature ranges within specific activities.

In summary, the comparative analysis highlights advancements and improvements in profiling activities compared to prior works. This approach distinguishes itself by enhanced accuracy, more labels through dataset integration, and pioneering behavior profiles. Challenges present valuable insights, laying the foundation for further advancements in DNS activity profiling.

Finally, the proposed model's effectiveness in identifying behavior patterns across various activities is confirmed. Distinctive behavior for each feature and varying activity correlations contribute to a robust profile. The introduction of DNS Flow revolutionizes behavior analysis, expanding the scope of the profiling system. These advancements bear implications for future research, emphasizing the crucial role of non-linearity in behavior patterns. This study validates the superiority of the proposed model, underscoring innovative contributions and paving the way for cutting-edge advancements in behavior analysis and classification.

8. Conclusion and future work

Creating dependable and openly accessible DNS evaluation datasets is crucial for both researchers and industry stakeholders. This paper evaluated the current landscape of DNS datasets and highlighted their shortcomings from various perspectives. Subsequently, a new DNS dataset named BCCC-CIC-Bell-DNS-2024 was introduced, resulting from integrating, cleaning, and enhancing two previous DNS datasets. A new concept termed DNS Flow was presented, along with a comprehensive feature set comprising over 120 features. Furthermore, ALFlowLyzer, an advanced application layer flow analyzer, was introduced to extract DNS flows and their corresponding feature sets.

Moreover, a novel graph-based feature selection algorithm was introduced, emphasizing the importance of incorporating common features alongside DNS-related ones for effectively profiling DNS activities. Additionally, a newly defined behavior similarity calculation metric was developed to enhance the accuracy of activity comparisons. This metric facilitated merging various DNS Exfiltration activities from previous datasets into a single general category in the new dataset. Moreover, a novel DNS behavior profiling model was designed to detect various DNS activities. This model ensures flexibility and interpretability in the generated profiles by leveraging a robust pattern extraction approach combined with a Neural Network structure.

Experimental results demonstrated the framework's ability to achieve an accuracy rate exceeding 99% in various profiling scenarios. The system detected various attacks, including Exfiltration and Spam activities, by analyzing key features.

Looking ahead, we aim to profile more DNS and non-DNS network activities by focusing on the application layer context. Continuous evaluation and refinement using diverse and larger-scale datasets will further validate and enhance the proposed profiling system's performance. Future research aims to advance behavior analysis and profiling, empowering organizations to better understand and mitigate security threats in an evolving network landscape.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: MohammadMoein Shafi reports financial support was provided by York University.

Data availability

The source code for ALFlowLyzer is publicly available in GitHub [23], and the dataset is downloadable from the BCCC website [31].

Acknowledgments

The authors acknowledge the Canada Research Chair - Tier II (#CRC-2021-00340) and the Natural Sciences and Engineering Research Council of Canada — NSERC (#RGPIN-2020-04701) — funds to Arash Habibi Lashkari.

References

- [1] Kasim Ömer. A robust DNS flood attack detection with a hybrid deeper learning model. *Comput Electr Eng* 2022;100:107883.
- [2] Shafi MohammadMoein, Lashkari Arash Habibi, Rodriguez Vicente, Nevo Ron. Toward generating a new cloud-based distributed denial of service (DDoS) dataset and cloud intrusion traffic characterization. *Information* 2024;15(4):195.
- [3] MahdaviFar Samaneh, Maleki Nasim, Lashkari Arash Habibi, Broda Matt, Razavi Amir H. Classifying malicious domains using DNS traffic analysis. In: 2021 IEEE intl conf on dependable, autonomic and secure computing, intl conf on pervasive intelligence and computing, intl conf on cloud and big data computing, intl conf on cyber science and technology congress. DASC/piCom/cBDCom/cyberSciTech, IEEE; 2021, p. 60–7.
- [4] MahdaviFar Samaneh, Hanafy Salem Amgad, Victor Princy, Razavi Amir H, Garzon Miguel, Hellberg Natasha, Lashkari Arash Habibi. Lightweight hybrid detection of data exfiltration using dns based on machine learning. In: 2021 the 11th international conference on communication and network security. 2021, p. 80–6.
- [5] Somarriba Oscar, Zurutuza Urko. A collaborative framework for android malware detection using DNS & dynamic analysis. In: 2017 IEEE 37th central america and panama convention. CONCAPAN XXXVII, IEEE; 2017, p. 1–6.
- [6] Dube Ishmael, Wells George. An analysis of the use of DNS for malicious payload distribution. In: 2020 2nd international multidisciplinary information technology and engineering conference. IMITEC, IEEE; 2020, p. 1–12.
- [7] Alieyan Kamal, Almoman Ammar, Anbar Mohammed, Alauthman Mohammad, Abdullah Rosni, Gupta Brij B. DNS rule-based schema to botnet detection. *Enterp Inf Syst* 2021;15(4):545–64.

- [8] Nguyen Thi Quynh, Laborde Romain, Benzekri Abdelmalek, Qu'hen Bruno. Detecting abnormal DNS traffic using unsupervised machine learning. In: 2020 4th cyber security in networking conference. CSNET, IEEE; 2020, p. 1–8.
- [9] Liu Zhenyan, Zeng Yifei, Zhang Pengfei, Xue Jingfeng, Zhang Ji, Liu Jiangtao. An imbalanced malicious domains detection method based on passive DNS traffic analysis. Secur Commun Netw 2018;2018.
- [10] Jiang Kui, Wang Fei. Detecting DNS tunnel based on multidimensional analysis. In: 2020 5th international conference on mechanical, control and computer engineering. ICMCE, IEEE; 2020, p. 272–5.
- [11] Zou Futai, Ren Yundong, Zhu Jiachen, Tang Junhua. Detecting data leakage in DNS traffic based on time series anomaly detection. In: 2021 IEEE 23rd int conf on high performance computing & communications; 7th int conf on data science & systems; 19th int conf on smart city; 7th int conf on dependability in sensor, cloud & big data systems & application. HPCC/DSS/smartCity/dependSys, IEEE; 2021, p. 503–10.
- [12] Lambion Danielle, Josten Michael, Olumofin Femi, De Cock Martine. Malicious DNS tunneling detection in real-traffic DNS data. In: 2020 IEEE international conference on big data. big data, IEEE; 2020, p. 5736–8.
- [13] Sharma Rohini, Guleria Ajay, Singla RK. Flow-based profile generation and network traffic detection for DNS anomalies using optimised entropy-based features selection and modified holt winter's method. Int J Secur Netw 2021;16(4):244–57.
- [14] Dwyer Owen P, Marnerides Angelos K, Giotsas Vasileios, Mursch Troy. Profiling iot-based botnet traffic using dns. In: 2019 IEEE global communications conference. GLOBECOM, IEEE; 2019, p. 1–6.
- [15] Perdisci Roberto, Papastergiou Thomas, Alrawi Omar, Antonakakis Manos. Iotfinder: Efficient large-scale identification of iot devices via passive dns traffic analysis. In: 2020 IEEE European symposium on security and privacy. euroS&p, IEEE; 2020, p. 474–89.
- [16] Dufera Abdisa G, Liu Tiantian, Xu Jin. Regression models of pearson correlation coefficient. Stat Theory Relat Fields 2023;7(2):97–106.
- [17] Shah Kulin, Chen Sitan, Klivans Adam. Learning mixtures of gaussians using the ddpm objective. Adv Neural Inf Process Syst 2023;36:19636–49.
- [18] Jin Chi, Zhang Yuchen, Balakrishnan Sivaraman, Wainwright Martin J, Jordan Michael I. Local maxima in the likelihood of gaussian mixture models: Structural results and algorithmic consequences. Adv Neural Inf Process Syst 2016;29.
- [19] Castillo-Barnes Diego, Martínez-Murcia Francisco Jesús, Ramírez Javier, Górriz JM, Salas-Gonzalez Diego. Expectation–maximization algorithm for finite mixture of α -stable distributions. Neurocomputing 2020;413:210–6.
- [20] Shawkat Mai, Badawi Mahmoud, El-ghamrawy Sally, Arnous Reham, El-desoky Ali. An optimized FP-growth algorithm for discovery of association rules. J Supercomput 2022;1–28.
- [21] Papazoglou Georgios, Biskas Pandelis. Review and comparison of genetic algorithm and particle swarm optimization in the optimal power flow problem. Energies 2023;16(3):1152.
- [22] Bian Kewei, Priyadarshi Rahul. Machine learning optimization techniques: a survey, classification, challenges, and future research issues. Arch Comput Methods Eng 2024;1–25.
- [23] BCCC-ALFlowLyzer. Application layer flow analyzer. ALFlowLyzer, Behaviour-Centric Cybersecurity Center (BCCC); 2023, URL: <https://github.com/ahlashkari/ALFlowLyzer>. [Retrieved 10 April 2023].
- [24] Man Keyu, Qian Zhiyun, Wang Zhongjie, Zheng Xiaofeng, Huang Youjun, Duan Haixin. Dns cache poisoning attack reloaded: Revolutions with side channels. In: Proceedings of the 2020 ACM SIGSAC conference on computer and communications security. 2020, p. 1337–50.
- [25] Kovar Ryan, Herrald David, Brodsky James. Boss of the SOC (BOTS) Dataset Version 1. URL: <https://github.com/splunk/botsv1>.
- [26] Sharma Rohini, Singla RK, Guleria Ajay. A new labeled flow-based DNS dataset for anomaly detection: PUF dataset. Procedia Comput Sci 2018;132:1458–66.
- [27] Zhao Qing, Qin Shihong. Study on security of web-based database. In: 2008 IEEE Pacific-Asia workshop on computational intelligence and industrial application. Vol. 2, IEEE; 2008, p. 902–5.
- [28] Lison Pierre, Mavroeidis Vasileios. Neural reputation models learned from passive DNS data. In: 2017 IEEE international conference on big data. big data, IEEE; 2017, p. 3662–71.
- [29] Jiang Jianguo, Chen Jiuming, Choo Kim-Kwang Raymond, Liu Chao, Liu Kunying, Yu Min, Wang Yongjian. A deep learning based online malicious URL and DNS detection scheme. In: Security and privacy in communication networks: 13th international conference, secureComm 2017, Niagara Falls, on, Canada, October 22–25, 2017, proceedings 13. Springer; 2018, p. 438–48.
- [30] Shi Yong, Chen Gong, Li Juntao. Malicious domain name detection based on extreme machine learning. Neural Process Lett 2018;48(3):1347–57.
- [31] BCCC-Dataset. BCCC-CIC-Bell-DNS-2024. Behaviour-Centric Cybersecurity Center (BCCC), URL: <https://www.yorku.ca/research/bccc/ucs-technical/cybersecurity-datasets-cds/>.

MohammadMoein Shafi is a graduate student pursuing a Master's degree in Computer Science at York University. Holding a bachelor's degree in Computer Engineering from the University of Tehran, MohammadMoein has nurtured a profound passion for the realms of Cybersecurity, Computer Networks, the Internet of Things, Machine Learning, Network Analysis, and Attack Analysis.

Arash Habibi Lashkari an Associate Professor and Canada Research Chair (CRC) in Cybersecurity, is the founder and director of the Behaviour-Centric Cybersecurity Center (BCCC) and brings over 26 years of teaching and research expertise. Recognized for his achievements, he has received 15 awards in international computer security competitions and was acknowledged as one of Canada's Top 150 Researchers.

Hardhik Mohanty was a Mitacs global internship researcher at the School of Information Technology at York University, Toronto, Canada. He is now a Ph.D. student at the Viterbi School of Engineering at the University of Southern California, Los Angeles, USA. His research interests include Blockchain and Decentralized Finance.